

Multimedia Retrieval - Report

Rutger Vincken¹, Bar Melinarskiy¹, and Arie Klaver¹

¹ Graduate School of Natural Sciences, Utrecht University, the Netherlands

September 16, 2025

Abstract

TBD

1 Introduction

Content-based 3D shape retrieval addresses the fundamental challenge of identifying similar three-dimensional objects within large databases using geometric and topological properties rather than textual metadata. Given a query shape, such systems must rank database entries by similarity and return the most relevant matches, requiring sophisticated understanding of shape characteristics across different scales and representations.

In this work, we implement a complete 3D shape retrieval system using a collection of triangular meshes spanning diverse semantic categories and geometric complexities. Originating from the Princeton Shape Benchmark [1], the dataset includes everyday objects, vehicles, buildings, tools, and animal models, offering a broad test-suite for evaluating retrieval accuracy and computational performance.

The system was developed through six main stages: data visualization and preprocessing, geometric normalization, feature extraction, similarity metric design, spatial indexing, and performance evaluation. Each stage builds on the previous to form an end-to-end pipeline that allows querying a shape and retrieving the most similar objects from the database. The following sections describe these stages in detail and present the results of our evaluation.

2 Dataset

TBD

3 Step 1: Data Exploration and Interactive Visualization Framework

Effective retrieval requires a clear understanding of the dataset at hand. To address this, our system begins with the development of an exploration and visualization framework tailored to 3D meshes. The following subsections provide a detailed breakdown of the application's implementation.

The purpose of this stage is to establish a comprehensive environment for systematic analysis of the mesh datasets. Interactive visualization is a central component in multimedia retrieval, as it enables researchers to assess data distributions, identify potential preprocessing challenges, and validate algorithmic decisions.

3.1 Framework Architecture and Design Rationale

The viewer app was developed as a web-based application, using Dash [2] to manage reactive interactions and Plotly [3] to provide efficient 3D visualization capabilities. The selection of this technology stack was guided by three requirements:

1. Real-time interactive exploration of the large mesh dataset.
2. Cross-platform accessibility without the need for specialized 3D software.
3. Seamless integration with Python-based scientific computing tools for efficient data processing, such as Pandas [4] and NumPy [5].

The framework architecture follows a model–view–controller paradigm [6], which separates data processing (model), user interface (view), and interaction logic (controller). This separation improves maintainability and supports extensibility for the integration of subsequent retrieval components.

3.2 Data Format Processing and Geometric Representation

Our system processes triangular mesh data stored in the Wavefront OBJ format, which has become the de facto standard for 3D model interchange due to its simplicity and widespread software support [7]. The OBJ format represents 3D objects through two primary components: vertex coordinates defining spatial geometry and face definitions specifying triangular connectivity.

We implemented a custom `OBJParser` class that addresses several technical challenges inherent in OBJ processing:

- **Index Convention Translation:** OBJ files use 1-based indexing while NumPy arrays require 0-based
- **Polygon Triangulation:** Automatic conversion of quad faces to triangles through diagonal splitting, ensuring consistent triangular mesh representation required for downstream processing
- **Robust Parsing:** Error handling for malformed geometry definitions and missing vertex references

The parser extracts vertex coordinates $V = \{v_1, v_2, \dots, v_n\}$ where $v_i \in \mathbb{R}^3$ and face connectivity $F = \{f_1, f_2, \dots, f_m\}$ where each face f_j contains three vertex indices, forming the fundamental mesh representation $M = (V, F)$ used throughout our retrieval pipeline.

3.3 Interactive Visualization System

The visualization component leverages Plotly’s WebGL-based 3D rendering engine, which provides hardware-accelerated graphics suitable for real-time interaction with complex meshes [8]. Our implementation incorporates several advanced rendering techniques:

To visualize 3D shapes in our web application, we use the `textitMesh3d` object from the Plotly Graph Objects library [9]. This tool allows us to render interactive 3D meshes directly in the browser, providing smooth rotation, zooming, and lighting effects. By leveraging Plotly’s capabilities, we can display complex surfaces and wireframes in a user-friendly way.

3.3.1 Wireframe Edge Visualization

Our optional wireframe overlay uses an efficient edge extraction method that avoids duplicate edges by storing them in a set. For each face f in the set of faces F , we iterate over its edges. Each edge is represented by a pair of vertex indices, sorted to ensure consistency. We add each sorted edge to the set E , which automatically removes duplicates.

This set-based approach guarantees $O(F)$ complexity for edge extraction, where F is the number of faces. The resulting set contains E unique edges. This significantly reduces rendering overhead compared to naive enumeration of all face edges [10, pp. 17–19].

Algorithm 1: Efficient Wireframe Edge Extraction

Input: Set of faces F (each face is a list of vertex indices)

Output: Set of unique edges E

```
1  $E \leftarrow \emptyset$ ; // Initialize empty edge set
2 foreach  $face\ f \in F$  do
3   for  $i \leftarrow 0$  to  $|f| - 1$  do
4      $v_1 \leftarrow f[i]$ ;
5      $v_2 \leftarrow f[(i + 1) \bmod |f|]$ ;
6      $edge \leftarrow \text{sort}(v_1, v_2)$ ; // Sort vertex indices for consistency
7      $E \leftarrow E \cup \{edge\}$ ; // Add edge to set (deduplication)
8   end
9 end
10 return  $E$ 
```

3.4 3D Shape Viewer Web Application

Our interactive web application, shown in Fig. 1, enables users to explore 3D models directly in their browser. The figure displays an example visualization of a bird mesh. Below, we describe the main components and design choices that support usability, performance, and data quality.

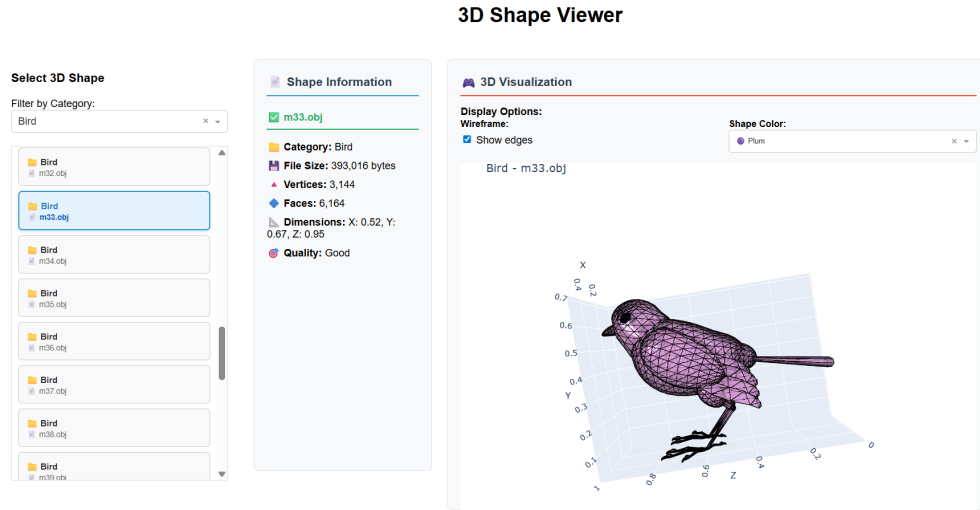


Figure 1: **Initial version of interactive user interface for 3D shape retrieval.** Users can filter the object list by category, such as “Bird,” or scroll through the complete list. Selecting a model reveals detailed information, including file size, vertices, faces, dimensions, and quality, along with an interactive 3D visualization. Display options such as wireframe edges and customizable colors support both structural inspection and visual comparison of shapes.

3.5 Hierarchical Data Organization and Metadata Extraction

To facilitate systematic exploration and evaluation, we utilized the organization of the 3D models into a hierarchical directory structure by category. The application automatically scans these folders, identifies available models, and extracts key metadata such as vertex and face counts, file size, and bounding box dimensions. This metadata is presented to users for quick assessment and is also used to label models as high or low resolution, supporting both user selection and algorithmic preprocessing. As shown in Fig. 2, users can filter models by category to quickly focus on the objects they wish to view.

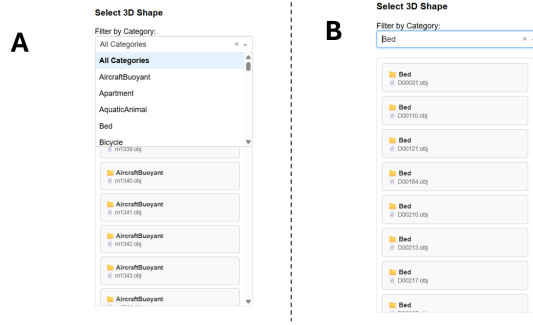


Figure 2: **Filter by category.** (A) The user can filter by a specific category or display objects from all categories. (B) For example, we filtered on all objects of beds.

3.6 Performance and Usability

To ensure a smooth user experience, our application is optimized for performance and reliability. Model files are loaded and parsed for display only when selected, which keeps memory usage low and startup times fast. Client-side handling of simple interactions, such as file selection highlighting, provides immediate feedback. Robust error handling further enhances usability by notifying users of any issues during file loading.

3.7 3D Visualization and Display Features

The viewer enables users to interactively explore 3D models using Plotly’s **Mesh3d** object. Users can rotate, zoom, and pan the models, as well as customize display options such as mesh color and wireframe edges. These features support both visual comparison and structural inspection, making it easier to understand the geometry and quality of each shape. These capabilities are demonstrated in Figures 3 and 4.

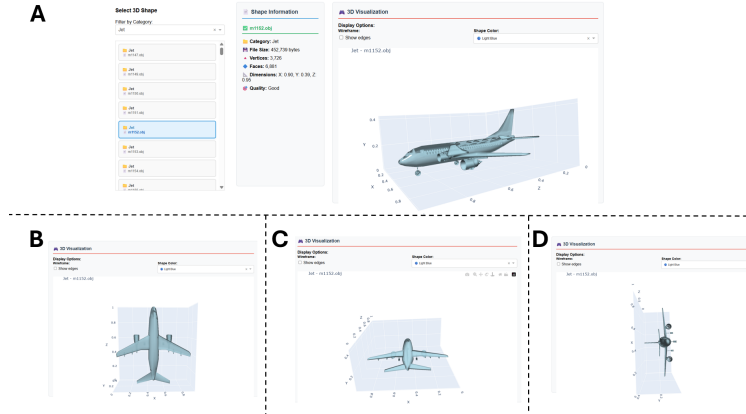


Figure 3: **Rotation demonstration.** (A) Example display of a plane object and its metadata; (B–D) the same object shown at different rotation angles.

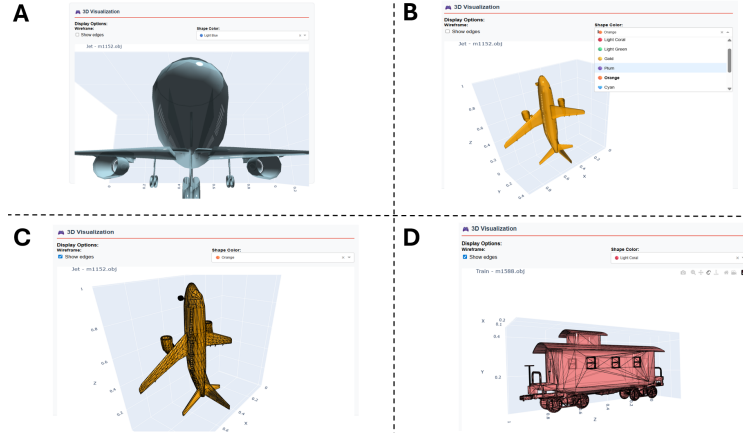


Figure 4: **Display features demonstration.** (A) Zooming in on the same plane object from Fig. 3. (B) Changing the object’s color from blue to orange. (C) Showing the plane’s edges by enabling the ”Show edges” checkbox above the viewer. (D) Displaying a pink train object with its edges visible, illustrating both color change and edge visualization.

This application streamlines the process of inspecting and comparing 3D models, providing both visual feedback and essential metadata to support further analysis and retrieval experiments.

3.8 3D Shape Viewer Web Application

4 Step 2: Preprocessing and cleaning

5 Step 3: Feature extraction

6 Step 4: Querying

7 Step 5: Scalability

8 Step 6: Evaluation

References

- [1] Philip Shilane et al. “The Princeton Shape Benchmark”. In: *Proceedings of the Shape Modeling International Conference*. IEEE, 2004, pp. 167–178. DOI: [10.1109/SMI.2004.1314504](https://doi.org/10.1109/SMI.2004.1314504). URL: <https://shape.cs.princeton.edu/benchmark/>.
- [2] Plotly Technologies Inc. *Dash: A web application framework for Python*. <https://plotly.com/dash>. Version 2.0 and above. 2017.
- [3] Plotly Technologies Inc. *Plotly.py: An interactive, open-source, and browser-based graphing library for Python*. <https://plotly.com/python>. Version 5.0 and above. 2015.
- [4] Wes McKinney et al. “pandas: a foundational Python library for data analysis and statistics”. In: *Python for high performance scientific computing*. Vol. 14. 9. Austin, TX. 2011, pp. 1–9.
- [5] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [6] John Deacon. “Model-view-controller (mvc) architecture”. In: *Online* [Citado em: 10 de março de 2006.] <http://www.jdl.co.uk/briefings/MVC.pdf> 28 (2009), p. 61.
- [7] Paul Bourke. *Object Files (.obj)*. <http://paulbourke.net/dataformats/obj/>. Accessed: December 2024. 1999.

- [8] Rufu Qin et al. “Web-based 3D visualization framework for time-varying and large-volume oceanic forecasting data using open-source technologies”. In: *Environmental Modelling & Software* 135 (2021), p. 104908.
- [9] Plotly Technologies Inc. *Plotly Graph Objects Python Library*. 2025. URL: <https://plotly.com/python/graph-objects/>.
- [10] Mario Botsch et al. *Polygon Mesh Processing*. CRC Press, 2010. URL: <https://www.pmp-book.org/>.